CERTIFICATION PROFESSIONNELLE
N° 36921
Expert en ingénierie des données

MSPR TPRE843 — Bloc 3
Plateforme Big Data sécurisée

GoodAir Platform

Conception et mise en œuvre d'une plateforme Big Data

Collecte — Stockage — Traitement distribué de données environnementales

> **Sujet (MSPR TPRE843) :**
> Le projet consiste à concevoir une architecture Big Data permettant d'ingérer, stocker et transformer des données environnementales à grande échelle. La plateforme mise en place vise à garantir la traçabilité des données, la robustesse des traitements et la disponibilité d'un entrepôt analytique exploitable par des outils de visualisation.

Groupe	Groupe 3
Équipe	Kenza Lamrani
	Yu-Ting HUANG
	Karl Evan MOUSSAVOU
	Komlavi ZODJRAKPE

Année académique	2024–2025

Table des matières

1 Contexte et enjeux du projet Big Data **3**
1.1 Présentation de TotalGreen et du laboratoire GoodAir . . . 3
1.2 Description générale du projet GoodAir Data Platform . . . 4
1.3 Analyse fonctionnelle et expression du besoin . . . 4
1.3.1 Matrice SWOT . . . 4
1.3.2 Analyse bête à cornes . . . 5
1.3.3 Cahier des charges fonctionnel . . . 5
1.3.4 Acteurs de la plateforme . . . 7
1.3.5 User Stories . . . 8
1.3.6 Cas d'utilisation . . . 9
1.3.7 Correspondance entre exigences fonctionnelles et composants techniques . . . 10

2 Architecture du pipeline GoodAir **11**
2.1 Vue d'ensemble de l'architecture du pipeline GoodAir . . . 11
2.1.1 Architecture logique . . . 11
2.1.2 Architecture technique et composants . . . 11
2.1.3 Technologies principales . . . 12
2.1.4 Schéma d'architecture globale . . . 12
2.2 Architecture du pipeline de données . . . 12
2.2.1 Justification du choix ELT . . . 13
2.2.2 Rôle d'Apache Spark dans l'architecture . . . 13
2.3 Fonctionnement Opérationnel du pipeline (de bout en bout) . . . 13
2.3.1 Environnement d'Exécution et Infrastructure Conteneurisée . . . 13
2.3.2 Sources de données et périmètre de collecte . . . 13
2.3.3 Orchestration du pipeline : Apache Airflow . . . 14
2.3.4 Étape 1 — Ingestion des données brutes (RAW) . . . 14
2.3.5 Étape 2 — Transformation et alimentation de la couche Silver . . . 14
2.3.6 Exposition des données et exploitation analytique . . . 15
2.4 Justification des choix technologiques . . . 16
2.4.1 Docker et docker-compose . . . 16
2.4.2 HDFS comme Data Lake . . . 17
2.4.3 Apache Spark pour la transformation des données . . . 17
2.4.4 Apache Airflow pour l'orchestration . . . 17
2.4.5 PostgreSQL comme entrepôt analytique . . . 17
2.4.6 Supervision et observabilité : Prometheus et Grafana et Alertmanager . . . 18
2.4.7 Python et bibliothèques de collecte . . . 18

3 Modélisation et description des tables **19**
3.0.1 Table `dim_city` . . . 19
3.0.2 Table `air_quality_current` . . . 19
3.0.3 Table `weather_current` . . . 19
3.0.4 Table `air_quality_forecast` . . . 20
3.1 Organisation du modèle analytique . . . 20
3.1.1 Principe du schéma . . . 20
3.1.2 Granularité des faits . . . 20
3.1.3 Avantages du modèle . . . 21

4 Gestion des logs et supervision du pipeline **22**
4.1 Journalisation des traitements (Logs) . . . 22
4.1.1 Logs d'orchestration (Airflow) . . . 22
4.1.2 Logs applicatifs : fonctionnels et techniques . . . 22
4.2 Traçabilité et auditabilité des données . . . 23
4.3 Métriques et supervision technique (Prometheus / Grafana) . . . 23
4.3.1 Collecte des métriques : rôle de Prometheus . . . 23
4.3.2 Exemples de métriques pertinentes . . . 23
4.3.3 Visualisation : rôle de Grafana . . . 23
4.4 Alerting et gestion proactive des incidents . . . 24
4.4.1 Règle d'alerte sur les échecs de tâches Airflow . . . 24
4.4.2 Rôle d'Alertmanager . . . 24

Conclusion **26**
Synthèse des réalisations et conformité au Bloc 3 (TPRE843) . . . 26
Ouverture et évolutivité du système . . . 26

Chapitre 1

Contexte et enjeux du projet Big Data

1.1 Présentation de TotalGreen et du laboratoire GoodAir

TotalGreen est un acteur français majeur dans le domaine des énergies renouvelables et de l'innovation environnementale. Engagée dans une stratégie de transition énergétique et de développement durable, l'entreprise accorde une place centrale à l'exploitation des données pour soutenir ses activités de recherche et d'aide à la décision.

Dans cette dynamique, TotalGreen a créé un laboratoire interne dédié à l'analyse environnementale : **GoodAir**. Cette entité scientifique a pour vocation de produire des connaissances fiables et exploitables sur l'évolution de la qualité de l'air et des conditions météorologiques sur le territoire français, en s'appuyant sur des données massives et hétérogènes.

Les missions du laboratoire GoodAir s'articulent autour de plusieurs axes stratégiques :

- assurer une surveillance continue et standardisée de la qualité de l'air dans les principales zones urbaines ;
- analyser l'impact des phénomènes climatiques sur la pollution atmosphérique ;
- fournir à la R&D des jeux de données structurés, historisés et prêts à être analysés ;
- soutenir la prise de décision grâce à des indicateurs fiables, reproductibles et scientifiquement documentés.

Dans un contexte marqué par l'accroissement des enjeux climatiques et réglementaires, la capacité du laboratoire GoodAir à exploiter efficacement les données constitue un levier essentiel de performance. Pour répondre à ces besoins, il est nécessaire de disposer d'une plateforme Big Data robuste, automatisée et scalable, capable de gérer :

- la collecte régulière de données issues de multiples sources externes ;
- leur historisation dans un Data Lake garantissant traçabilité et intégrité ;
- leur transformation selon des standards industriels ;
- leur mise à disposition dans un format exploitable pour des analyses avancées.

Le présent projet s'inscrit précisément dans cette démarche de structuration et d'industrialisation des processus de traitement de données environnementales. Il constitue un socle technique destiné à renforcer les capacités d'analyse du laboratoire GoodAir et à accompagner TotalGreen dans ses ambitions scientifiques et stratégiques.

Pour répondre à ces besoins, une plateforme de données a été conçue autour d'un pipeline automatisé basé sur des technologies open source : Docker, Apache Airflow, Apache Spark, HDFS, PostgreSQL et Metabase. Cette architecture, modulaire et extensible, constitue la base technique du projet **GoodAir Data Platform**.

1.2 Description générale du projet GoodAir Data Platform

Le projet **GoodAir Data Platform** vise la construction d'une chaîne de traitement de données complète, fiable et reproductible autour des axes suivants :

- **Collecte des données** : interrogation périodique des API publiques WAQI (World Air Quality Index) et OpenWeather pour une vingtaine de villes françaises ;
- **Stockage brut (*raw*)** : archivage des réponses JSON dans un **Data Lake HDFS**, garantissant une conservation intégrale, horodatée et non altérée ;
- **Transformation (*silver*)** : normalisation, nettoyage, enrichissement et structuration des données à l'aide de **Apache Spark** pour alimenter un modèle relationnelle exploitable ;
- **Chargement en base** : intégration des données standardisées dans un **entrepôt relationnel PostgreSQL** permettant l'interrogation SQL, l'analyse statistique et la construction de visualisations ;

Cette plateforme constitue ainsi une fondation technique robuste permettant à GoodAir de centraliser son patrimoine de données environnementales et de faciliter les travaux de recherche. Elle repose sur un écosystème Big Data open source, dont les choix technologiques seront détaillés dans les chapitres suivants.

1.3 Analyse fonctionnelle et expression du besoin

Afin d'évaluer les enjeux et les risques associés à la mise en place de la plateforme, une analyse SWOT a été réalisée. Celle-ci permet d'identifier les forces, faiblesses, opportunités et menaces du projet.

1.3.1 Matrice SWOT

La matrice SWOT permet d'analyser de manière synthétique les forces, faiblesses, opportunités et menaces associées au projet GoodAir Data Platform.

Cette analyse prend en compte les aspects organisationnels, techniques et contextuels afin d'identifier les leviers de succès et les risques potentiels du projet.

Forces (Strengths)	**Faiblesses (Weaknesses)**
— Utilisation de données publiques fiables et accessibles ; — Centralisation de données auparavant dispersées ; — Outil favorisant la compréhension et la sensibilisation sur la qualité de l'air.	— Dépendance aux données publiques externes ; — Contrôle limité sur la qualité et la fréquence de mise à jour des données ; — Absence de modèle économique direct ; — Besoin de compétences techniques pour la maintenance et l'évolution.
Opportunités (Opportunities)	**Menaces (Threats)**
— Intérêt croissant pour la qualité de l'air dans les collectivités, médias et citoyens ; — Possibilité de collaboration avec des organismes publics, ONG ou instituts scientifiques ; — Demandes grandissantes pour des outils de visualisation environnementale ; — Peut devenir un outil d'aide à la décision pour les villes dans leurs politiques environnementales.	— Indisponibilité ou modification des APIs externes, limitant l'accès aux données ; — Variabilité de la qualité des données environnementales ; — Contraintes réglementaires (RGPD, confidentialité, hébergement des données) ; — Changements possibles dans les politiques publiques d'ouverture des données ; — Concurrence d'observatoires ou plateformes publiques déjà existants.

Table 1.1 – Analyse SWOT du projet GoodAir Data Platform

1.3.2 Analyse bête à cornes

Pour mieux comprendre le besoin et le public cible de notre plateforme, nous avons réalisé un bête à cornes qui met en évidence les éléments essentiels du projet 1.2

Nom du projet	GoodAir Data Platform
Utilisateur final	Chercheurs, analystes climat/météo, responsables R&D
Sources	APIs WAQI (qualité de l'air) et OpenWeather (météo)
Objectif	Centraliser, historiser, analyser et visualiser les données environnementales pour soutenir la recherche.

Table 1.2 – Bête à cornes du projet GoodAir

1.3.3 Cahier des charges fonctionnel

Le cahier des charges fonctionnel formalise les exigences attendues pour la plateforme **GoodAir Data Platform**. L'objectif est de garantir une collecte automatisée, une conservation historisée, une transformation reproductible et une mise à disposition simple des données environnementales (qualité de l'air et météo) à des fins d'analyse.

Périmètre fonctionnel

La plateforme couvre les fonctions suivantes :

- ingestion automatique des données depuis les API **WAQI** et **OpenWeather** ;
- stockage des réponses brutes (JSON) dans un **Data Lake HDFS** partitionné par date/-heure ;
- transformation et normalisation des données via **Spark / PySpark** ;
- chargement dans un **entrepôt PostgreSQL** selon un schéma relationnel exploitable ;
- historisation des mesures et traçabilité technique (horodatage, source, exécution) ;
- accès aux données via requêtes SQL et outil de visualisation (ex. Metabase).

Exigences fonctionnelles

EF1 — Collecte automatisée des données

- La plateforme doit interroger périodiquement les API WAQI et OpenWeather.
- La collecte doit couvrir un ensemble de villes (environ 20 villes françaises).
- Chaque appel API doit être tracé (heure d'appel, statut HTTP, succès/échec).

EF2 — Stockage brut et historisation dans le Data Lake

- Les réponses brutes doivent être stockées au format JSON sans altération.
- Les données doivent être historisées et conservées, même en cas d'évolution du schéma API.
- Le stockage doit être partitionné par `dt=YYYY-MM-DD` et `hour=HH` afin de faciliter la recherche et le recalcul (*reprocessing*).

EF3 — Transformation et normalisation des données

- Les données brutes doivent être nettoyées : suppression des doublons, gestion des valeurs nulles, contrôle des types, standardisation des noms.
- Les données doivent être structurées selon des tables cibles : `dim_city`, `air_quality_current`, `weather_current`, `air_quality_forecast`.
- La transformation doit garantir la cohérence temporelle (conversion timezones, timestamps).

EF4 — Alimentation du Data Warehouse PostgreSQL

- Les données normalisées doivent être chargées dans PostgreSQL en mode incrémental.
- La plateforme doit éviter l'insertion de doublons grâce à des clés techniques (*ex.* `city_id + fetched_at_utc`).
- La dimension des villes (`dim_city`) doit être alimentée automatiquement à partir des sources RAW (WAQI + OpenWeather).

EF5 — Consultation et exploitation des données

- Les données doivent être accessibles via des requêtes SQL simples.
- La structure doit être documentée et compréhensible pour un Data Analyst.
- Un outil de BI (Metabase) doit pouvoir se connecter et exploiter les tables sans transformation supplémentaire.

EF6 — Supervision et fiabilité opérationnelle

- Chaque exécution doit produire des logs exploitables (Airflow + Spark).
- Le système doit permettre la relance d'un pipeline en cas d'échec.
- Des métriques techniques doivent être disponibles (CPU, mémoire, disponibilité des services) via Prometheus et Grafana.

Contraintes et exigences non fonctionnelles

ENF1 — Robustesse et reprise

- Le pipeline doit gérer les indisponibilités temporaires des API (timeouts, codes d'erreur).
- La plateforme doit permettre un recalcul à partir du RAW (*reprocessing*) pour une date/heure donnée.

ENF2 — Performance

- Les traitements doivent être capables de traiter au minimum 20 villes par heure.
- Les opérations de transformation doivent être parallélisables et optimisées (Spark).

ENF3 — Traçabilité et auditabilité

- Chaque donnée chargée doit être associée à un horodatage de mesure (`measured_at`) et à un horodatage de collecte (`fetched_at_utc`).
- Les sources et versions de traitements doivent pouvoir être identifiées (logs, code versionné).

ENF4 — Sécurité et conformité

- Les services doivent être isolés dans un environnement conteneurisé (Docker).
- L'accès aux bases et outils doit être restreint (identifiants PostgreSQL, accès Metabase).
- Les données traitées ne contiennent pas de données personnelles ; la plateforme respecte donc les principes généraux du RGPD (minimisation, traçabilité, séparation des accès).

Synthèse des livrables attendus

Les livrables fonctionnels et techniques attendus sont :

- une architecture conteneurisée opérationnelle (Docker Compose) ;
- un pipeline Airflow orchestrant l'ingestion RAW et les transformations Silver ;
- un Data Lake HDFS structuré et historisé ;
- un modèle relationnel PostgreSQL cohérent et documenté ;
- un outil de visualisation connecté (Metabase) ;
- des éléments de supervision (logs, métriques Prometheus, dashboards Grafana).

1.3.4 Acteurs de la plateforme

La plateforme **GoodAir Data Platform** est exploitée par plusieurs acteurs techniques et métiers, intervenant à différents niveaux du cycle de vie de la donnée. Ces acteurs contribuent au bon fonctionnement du pipeline, à la qualité des données produites et à leur valorisation analytique.

Back-office

Le Data Engineer / Administrateur de la plateforme Le Data Engineer est responsable de la conception, du déploiement et de la maintenance de l'infrastructure technique permettant la collecte, l'historisation et le traitement des données environnementales.

Il garantit la fiabilité du pipelin et la continuité des traitements automatisés. Ses principales missions sont les suivantes :

- superviser l'exécution des pipelines de données ;
- assurer le bon fonctionnement des traitements de transformation distribués ;
- garantir la disponibilité et la cohérence des services techniques de la plateforme ;
- surveiller les performances et l'état global du système ;
- gérer les incidents liés aux sources de données externes, aux erreurs de traitement ou aux interruptions de service ;
- veiller à l'alimentation régulière, cohérente et historisée du Data Warehouse.

Le Data Engineer assure ainsi la stabilité globale de la plateforme et la mise à disposition de données fiables, prêtes à être exploitées par les équipes d'analyse.

Le Data Analyst Le Data Analyst exploite les données collectées et transformées par le pipelin. Il intervient principalement sur la couche analytique du système et se concentre sur l'interprétation, le contrôle et la valorisation des données.

Ses missions principales incluent :

- accéder aux données nettoyées, normalisées et historisées du Data Warehouse ;
- analyser la cohérence et la continuité des séries temporelles ;
- détecter d'éventuelles anomalies, ruptures ou incohérences ;
- produire des analyses, tableaux de bord ou rapports internes ;
- formuler des recommandations visant à améliorer la qualité ou la pertinence des données collectées.

Son rôle garantit que les données centralisées répondent aux besoins analytiques et scientifiques du laboratoire GoodAir.

1.3.5 User Stories

Afin de traduire les besoins fonctionnels de manière concrète, plusieurs *user stories* ont été définies pour les principaux profils utilisateurs.

User stories – Data Engineer / Administrateur

User story *En tant qu'administrateur de l'infrastructure, je souhaite superviser les pipelines et les traitements automatisés afin de garantir que la collecte, l'historisation et le stockage des données fonctionnent correctement et sans interruption.*

Objectif Assurer la fiabilité du pipeline ETL et la mise à disposition continue des données pour les chercheurs et analystes.

Tâches clés

- superviser l'état des pipelines et des services de la plateforme ;
- suivre l'exécution des traitements et identifier les incidents ;
- gérer les erreurs liées aux pipelines ou aux sources de données externes ;
- assurer l'alimentation continue du Data Warehouse.

Critères d'acceptation

- état des pipelines visible et mis à jour en temps réel ;
- alertes automatiques en cas d'échec de traitement ou d'absence de données ;
- historique des exécutions consultable ;
- possibilité de relancer un pipeline en erreur.

User stories – Data Analyst

User story *En tant que Data Analyst, je souhaite accéder à des données propres, historisées et centralisées afin de pouvoir réaliser des analyses fiables et reproductibles.*

Objectif Faciliter l'exploitation, l'analyse et la réutilisation des données produites par la plateforme GoodAir.

Tâches clés

— consulter les jeux de données disponibles dans le Data Warehouse ;
— vérifier la cohérence des séries temporelles ;
— identifier d'éventuelles anomalies ou ruptures ;
— produire des analyses et rapports à partir des données préparées.

Critères d'acceptation

— données accessibles via un schéma clair, documenté et stable ;
— historisation complète et cohérente des données ;
— absence de doublons ou d'incohérences majeures ;
— disponibilité des données dans les délais attendus.

1.3.6 Cas d'utilisation

Cette section présente les principaux cas d'utilisation identifiés pour la plateforme GoodAir Data Platform.

Superviser les pipelines

Description : Le Data Engineer supervise l'exécution des pipelines de collecte, de transformation et d'historisation des données afin d'assurer leur bon fonctionnement.

Données d'entrée : Aucune. L'état des pipelines est automatiquement fourni par le système.

Scénario nominal :

— le Data Engineer accède à l'interface de supervision ;
— le système affiche l'état des pipelines (succès, en cours, échec) ;
— le Data Engineer consulte les logs d'exécution ;
— l'état du pipeline est mis à jour en temps réel.

Remarque : Ce cas d'utilisation permet d'identifier rapidement les anomalies ou retards de traitement.

Gérer et relancer un pipeline en erreur

Description : Le Data Engineer analyse et corrige les erreurs rencontrées lors de l'exécution du flux.

Données d'entrée : Logs d'erreurs, messages de défaillance API, paramètres du job concerné.

Scénario nominal :

— le Data Engineer reçoit une notification d'échec ;
— il consulte les logs pour identifier la cause ;
— il applique les actions correctives nécessaires ;
— le pipeline est relancé manuellement ;
— le système confirme l'exécution avec succès.

Analyser et contrôler la qualité des données

Description : Le Data Analyst consulte les données préparées afin de réaliser des analyses et de contrôler leur qualité.

Données d'entrée : Tables historisées et normalisées du Data Warehouse.

Scénario nominal :

— le Data Analyst accède au Data Warehouse ;
— il sélectionne une table ou un jeu de données ;
— il vérifie la cohérence, l'exhaustivité et la validité des données ;
— les anomalies détectées sont signalées au Data Engineer si nécessaire.

Remarque : Ce cas d'utilisation garantit la fiabilité scientifique des données exploitées par le laboratoire GoodAir.

1.3.7 Correspondance entre exigences fonctionnelles et composants techniques

Le tableau suivant établit la correspondance entre les exigences fonctionnelles exprimées dans le cahier des charges et les composants techniques mis en œuvre au sein de la **GoodAir Data Platform**. Il permet de démontrer l'adéquation entre les besoins métier et les choix technologiques réalisés.

Table 1.3 – Correspondance entre exigences fonctionnelles et composants techniques

Exigence fonctionnelle	**Composants techniques associés**
Collecte automatisée des données environnementales depuis les API externes	Apache Airflow (orchestration) Python (scripts d'appel API) APIs WAQI et OpenWeather
Stockage brut et historisation complète des données	HDFS (Data Lake) Format JSON Partitionnement `dt/hour`
Transformation, nettoyage et normalisation des données	Apache Spark / PySpark Jobs batch horaires
Alimentation incrémentale du Data Warehouse	PostgreSQL JDBC Spark → PostgreSQL Clés techniques (`city_id`, `fetched_at_utc`)
Accès aux données prêtes à l'analyse	DBeaver,Metabase
Supervision et fiabilité opérationnelle du pipeline	Airflow (logs, DAGs, retries) Prometheus Grafana
Robustesse et reprise sur incident	Historisation RAW Reprocessing par date/heure Relance manuelle Airflow
Performance et scalabilité	Spark (traitement distribué) Partitionnement HDFS Traitements incrémentaux
Traçabilité et auditabilité des données	Champs `measured_at`, `fetched_at_utc` Logs Airflow / Spark
Sécurité et conformité	Docker (isolation des services) Accès contrôlé PostgreSQL Données non personnelles (RGPD)

Chapitre 2

Architecture du pipeline GoodAir

2.1 Vue d'ensemble de l'architecture du pipeline GoodAir

La *GoodAir Data Platform* est conçue comme une plateforme Big Data reposant sur un **traitement batch** et une architecture modulaire, permettant de gérer l'intégralité du cycle de vie des données environnementales : de leur collecte jusqu'à leur exploitation analytique.

L'architecture du pipeline repose sur une organisation claire des rôles et des responsabilités des composants, afin de garantir :

— la traçabilité complète des données ;
— la séparation des phases de collecte, de stockage et de transformation ;
— la reproductibilité des traitements ;
— la facilité d'évolution de la plateforme.

La plateforme s'appuie exclusivement sur des technologies open source et est déployée dans un environnement conteneurisé, assurant une exécution homogène et maîtrisée.

2.1.1 Architecture logique

D'un point de vue logique, le pipeline GoodAir est structuré selon une chaîne de traitement séquentielle, orchestrée et automatisée, s'inscrivant dans un modèle **ELT**.

L'architecture s'articule autour de deux couches principales :

— **Couche Raw (Data Lake)** : destinée au stockage des données brutes issues des sources externes, sans transformation ;
— **Couche Silver (entrepôt analytique)** : destinée au stockage des données transformées, structurées et prêtes à l'analyse.

Cette séparation permet de conserver une vision claire entre les données sources et les données exploitables, tout en garantissant l'historisation complète des informations collectées.

2.1.2 Architecture technique et composants

L'architecture technique de la *GoodAir Platform* repose sur l'interconnexion de plusieurs briques technologiques, chacune répondant à un besoin spécifique du pipeline :

— des **APIs externes** pour la fourniture des données environnementales ;
— un **Data Lake** pour le stockage des données brutes ;
— un **moteur de traitement distribué** pour les transformations ;
— un **entrepôt analytique** pour la restitution des données ;
— des **outils d'orchestration et de supervision** pour piloter et surveiller la plateforme.

L'ensemble de ces composants est déployé sous forme de conteneurs Docker, orchestrés par `docker-compose`, dans un environnement Linux exécuté via **WSL**.

2.1.3 Technologies principales

Les principales technologies constituant le pipeline GoodAir sont :

- **Apache Airflow** pour l'orchestration des traitements batch ;
- **HDFS** pour le stockage des données brutes (Data Lake) ;
- **Apache Spark (PySpark)** pour exctraction et la transformation et la structuration des données ;
- **PostgreSQL** pour le stockage analytique des données transformées ;
- **Prometheus** pour la collecte des métriques techniques ;
- **Grafana** pour la visualisation et la supervision de la plateforme ;
- **Alertmanager** pour la gestion centralisée des alertes et la notification proactive ;
- **Metabase** pour l'analyse et la visualisation des données à destination des utilisateurs métier ;
- **Docker et docker-compose** pour le déploiement et l'isolation des services.

Ces technologies sont utilisées de manière complémentaire afin de constituer une plateforme cohérente, robuste et adaptée aux contraintes du projet.

2.1.4 Schéma d'architecture globale

L'architecture décrite ci-dessus est synthétisée dans le schéma présenté à la figure suivante. Ce diagramme illustre les différentes couches du pipeline, les flux de données entre les composants ainsi que les principales interactions entre les services.

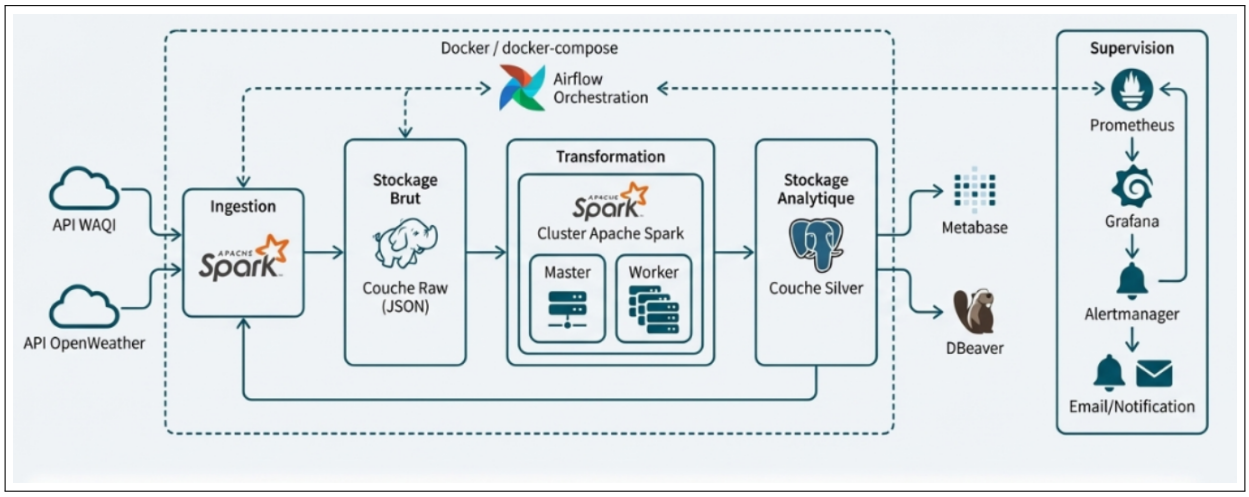

Figure 2.1 – Flux haut niveau du pipeline *GoodAir Platform*.

2.2 Architecture du pipeline de données

Le pipeline mis en œuvre suit une approche de type **ELT (Extract – Load – Transform)**. Ce choix architectural est volontaire et s'inscrit dans les pratiques modernes des plateformes Big Data industrielles.

Il est important de souligner que la distinction entre un pipeline ETL et ELT ne dépend pas des technologies utilisées (Apache Spark, calcul en mémoire, etc.), mais exclusivement de l'ordre logique dans lequel les étapes de traitement sont réalisées.

Dans le cas du projet GoodAir, les données sont d'abord extraites depuis les API, puis stockées telles quelles, sans modification, dans un **Data Lake HDFS** sous forme de fichiers JSON horodatés. Les transformations (nettoyage, typage,structuration) sont ensuite réalisées à partir de ces données brutes à l'aide d'Apache Spark.

Ce fonctionnement correspond strictement à un modèle ELT.

2.2.1 Justification du choix ELT

Le choix d'un pipeline ELT est justifié par plusieurs considérations techniques et fonctionnelles :

— **Conservation intégrale des données brutes** : Le stockage des réponses JSON dans HDFS permet de conserver une trace fidèle et immuable des données sources, garantissant la traçabilité et la conformité aux bonnes pratiques Big Data.

— **Rejouabilité des traitements** : Les transformations peuvent être relancées à tout moment à partir des données brutes, sans nécessiter de nouvel appel aux API externes.

— **Découplage entre collecte et transformation** : La phase d'extraction est isolée de la phase de transformation, ce qui améliore la robustesse globale du système et facilite la maintenance.

— **Scalabilité** : Le Data Lake HDFS permet de gérer un volume croissant de données sans remise en cause de l'architecture existante.

— **Évolution fonctionnelle facilitée** : De nouvelles règles de transformation ou de nouveaux modèles de données peuvent être appliqués ultérieurement sans impacter les données sources.

2.2.2 Rôle d'Apache Spark dans l'architecture

Apache Spark est utilisé comme moteur de transformation dans la phase **Transform** du pipeline ELT. Bien que Spark repose sur un moteur de calcul en mémoire, ce point n'influence pas la classification ETL ou ELT du pipeline.

Spark intervient exclusivement **après** le chargement des données brutes dans HDFS. Il lit les fichiers JSON stockés dans le Data Lake, applique les règles de transformation (flatten des structures JSON, nettoyage, typage) et produit des tables relationnelles prêtes à être exploitées.

Ainsi, Spark est un outil de transformation post-chargement, parfaitement compatible avec une architecture ELT moderne.

2.3 Fonctionnement Opérationnel du pipeline (de bout en bout)

Cette section décrit le fonctionnement réel de la *GoodAir Platform* en conditions d'exécution, depuis les sources de données externes jusqu'à la restitution analytique et la supervision technique. La plateforme est exécutée dans un environnement Linux via **WSL**, et l'ensemble des services est déployé sous forme de conteneurs Docker orchestrés par **docker-compose**.

2.3.1 Environnement d'Exécution et Infrastructure Conteneurisée

La *GoodAir Platform* est déployée dans un environnement Linux fourni par **WSL (Windows Subsystem for Linux)**. Tous les composants techniques et applicatifs sont encapsulés dans des conteneurs Docker afin de garantir l'isolation des services, la reproductibilité de l'environnement et la portabilité du pipeline.

L'orchestration des conteneurs repose sur le fichier `docker-compose.yml`, qui définit :

— les images Docker utilisées ;

— les dépendances entre services ;

— les volumes de persistance ;

— les ports exposés vers **localhost**.

2.3.2 Sources de données et périmètre de collecte

Le pipeline exploite deux APIs publiques complémentaires :

— **WAQI (World Air Quality Index)** : indices de qualité de l'air, polluants atmosphériques (PM2.5, PM10, NO_2, O_3, SO_2, CO) et prévisions ;

— **OpenWeather** : variables météorologiques (température, humidité, pression, vent, nébulo-

sité).

La collecte est réalisée pour un ensemble de villes françaises, selon une fréquence **horaire**, compatible avec les quotas des APIs et adaptée aux besoins analytiques du projet.

2.3.3 Orchestration du pipeline : Apache Airflow

L'orchestration du pipeline est assurée par Apache Airflow au moyen d'un DAG horaire nommé `goodair_hourly`. Ce DAG structure le traitement en deux étapes séquentielles :

1. ingestion des données brutes (RAW) ;
2. transformation et chargement des données (SILVER).

Cette organisation garantit que les traitements de transformation ne sont exécutés que si la collecte du créneau horaire a abouti avec succès.

Services d'orchestration : conteneurs Airflow

Airflow est déployé à l'aide de deux conteneurs distincts :

— **airflow-webserver** : interface de pilotage des DAGs et consultation des logs, accessible via `http://localhost:8080` ;

— **airflow-scheduler** : responsable de la planification et du déclenchement des tâches.

2.3.4 Étape 1 — Ingestion des données brutes (RAW)

La première tâche du DAG exécute le script `raw_ingest_paris.py`. Ce job interroge les APIs WAQI et OpenWeather, récupère les réponses JSON complètes et les stocke sans transformation dans la couche Raw du Data Lake.

Stockage Raw et partitionnement Les données sont stockées dans HDFS avec un partitionnement strict par date et heure, garantissant traçabilité et reprocessing ciblé :

```
/data/raw/
  waqi/dt=YYYY-MM-DD/hour=HH/
  openweather/dt=YYYY-MM-DD/hour=HH/
```

Service de stockage brut : HDFS

La couche Raw repose sur HDFS, déployé via deux conteneurs :

— **hadoop-namenode** : gestion des métadonnées (interface : `http://localhost:9870`) ;

— **hadoop-datanode** : stockage physique des blocs de données.

2.3.5 Étape 2 — Transformation et alimentation de la couche Silver

Après validation de la collecte, Airflow déclenche le job Spark `goodair_silver_hourly.py`. Ce job lit uniquement les partitions Raw correspondant au créneau horaire courant. Les traitements Spark incluent :

— parsing et structuration des données JSON ;

— nettoyage et normalisation des valeurs ;

— alignement temporel des données météo et air ;

— production de DataFrames exploitables.

Service de traitement : Apache Spark

Apache Spark est déployé sous forme d'un cluster minimal :

— **spark-master** : coordination des jobs (UI : `http://localhost:8081`) ;

— **spark-worker** : exécution des tâches distribuées.

Service de stockage analytique : PostgreSQL

Les données transformées sont chargées de manière incrémentale dans PostgreSQL (couche Silver), accessible via `localhost:5432`. Cette base constitue le point d'accès unique pour l'analyse.

2.3.6 Exposition des données et exploitation analytique

Les données de la couche Silver sont exposées via des outils de consultation et de visualisation connectés directement à PostgreSQL, permettant :

- l'exploration et la compréhension du modèle de données ;
- l'exécution de requêtes analytiques ;
- la création de tableaux de bord et d'indicateurs.

Outils d'analyse et de visualisation : DBeaver et Metabase

Deux outils complémentaires sont utilisés pour l'exploitation des données :

- **DBeaver** : client SQL permettant l'exploration des schémas, l'exécution de requêtes et les contrôles techniques sur les données ;
- **Metabase** : outil de visualisation orienté métier, utilisé pour créer des tableaux de bord interactifs à partir des données PostgreSQL.

Supervision technique : Prometheus, Grafana et Alertmanager

La supervision de la plateforme repose sur les composants suivants :

- **Prometheus** : collecte et stockage des métriques techniques (interface : `http://localhost:9090`) ;
- **Grafana** : visualisation des métriques et supervision globale à travers des tableaux de bord (interface : `http://localhost:3001`) ;
- **Alertmanager** : gestion centralisée des alertes et envoi de notifications automatiques en cas de détection d'anomalies.

Table 2.1 – Synthèse du fonctionnement de bout en bout du pipeline GoodAir

Étape	Composant	Rôle principal	Résultat produit
1	Apache Airflow	Déclenchement et orchestration du DAG horaire `goodair_hourly`	Pilotage et traçabilité des exécutions
2	APIs WAQI & OpenWeather	Collecte des données environnementales (qualité de l'air et météo)	Réponses JSON brutes
3	HDFS (Couche Raw)	Stockage des données brutes, partitionnées par date et heure	Fichiers JSON historisés
4	Apache Spark (PySpark)	Transformation, nettoyage et normalisation des données	DataFrames structurés
5	PostgreSQL (Couche Silver)	Chargement incrémental et stockage analytique	Tables prêtes à l'analyse
6	Metabase	Exploration et visualisation des données	Tableaux de bord analytiques
7	Prometheus, Grafana & Alertmanager	Collecte des métriques, visualisation et alerting automatisé	Dashboards de supervision et notifications d'alerte

2.4 Justification des choix technologiques

L'architecture de la *GoodAir Platform* repose sur un ensemble de technologies open source reconnues dans l'écosystème Big Data. Les choix effectués visent à garantir la **robustesse**, la **traçabilité**, la **scalabilité** et la **maintenabilité** de la plateforme.

2.4.1 Docker et docker-compose

L'ensemble des composants de la plateforme (Apache Airflow, Apache Spark, HDFS, PostgreSQL, Grafana et Prometheus) a été conteneurisé à l'aide de **Docker**. Le démarrage et la configuration des services sont orchestrés via **docker-compose**.

Ce choix présente plusieurs avantages majeurs :

— **Portabilité** : la plateforme peut être déployée de manière identique sur un poste de développement, un serveur pédagogique ou un environnement cloud ;

— **Isolation des services** : chaque composant dispose de ses dépendances et de sa configuration propre, limitant les conflits et facilitant le débogage ;

— **Reproductibilité** : l'infrastructure est décrite de manière déclarative dans des fichiers versionnés (`Dockerfile`, `docker-compose.yml`), garantissant la reproductibilité des environnements.

Cette approche permet également de simplifier l'exploitation de la plateforme : une seule commande (`docker-compose up`) suffit à lancer l'ensemble des services, y compris le Data Lake, les moteurs de traitement et les outils de supervision.

2.4.2 HDFS comme Data Lake

Les données brutes issues des APIs (WAQI et OpenWeather) sont stockées dans un **Data Lake HDFS**. Ce choix s'est imposé plutôt qu'un stockage local ou une base relationnelle, afin de répondre aux exigences de traçabilité et d'évolutivité.

HDFS permet notamment :

— le stockage de **volumes importants** de données semi-structurées (JSON) ;
— une gestion efficace des **schémas évolutifs** ;
— une organisation des données en **partitions temporelles** (date et heure), limitant les volumes relus lors des traitements.

Le Data Lake constitue ainsi une couche de référence, garantissant la **rejouabilité des traitements** et la conservation de l'historique des données collectées.

2.4.3 Apache Spark pour la transformation des données

Les traitements de nettoyage, de normalisation et d'enrichissement sont réalisés à l'aide d'**Apache Spark**, via son API **PySpark**.

Spark est utilisé pour :

— le parsing des structures JSON complexes retournées par les APIs ;
— la validation et le filtrage des données exploitables ;
— la normalisation des formats (horodatages, unités de mesure) ;
— la production de tables prêtes à l'analyse.

Le choix de Spark plutôt que des scripts Python *standalone* (Pandas) se justifie par :

— sa capacité à **passer à l'échelle** en cas d'augmentation du volume de données ;
— son intégration native avec HDFS et PostgreSQL ;
— la possibilité de déployer les mêmes traitements sur un cluster distribué réel sans refonte majeure du code.

2.4.4 Apache Airflow pour l'orchestration

L'orchestration du pipeline est assurée par **Apache Airflow**. Un DAG principal (`goodair_raw_hourly`) a été défini afin de piloter les différentes étapes du traitement.

Ce DAG :

— planifie la collecte des données sur une base horaire ;
— déclenche les jobs Spark de collecte et de transformation ;
— gère les dépendances entre les tâches ;
— centralise les journaux d'exécution et les états des traitements.

L'utilisation d'Airflow, plutôt qu'un simple ordonnanceur système (`cron`), apporte une **traçabilité complète**, une meilleure gestion des erreurs (*retries*, *failures*) et une forte capacité d'évolution du pipeline.

2.4.5 PostgreSQL comme entrepôt analytique

Les données transformées sont chargées dans un entrepôt analytique basé sur **PostgreSQL**, au sein d'un schéma dédié `goodair`.

Ce choix est motivé par :

— la maturité et la fiabilité de PostgreSQL en tant que SGBDR open source ;
— sa compatibilité avec les outils de visualisation et d'analyse ;
— la possibilité d'appliquer des contraintes d'intégrité (types, clés, *not null*) renforçant la qualité des données.

PostgreSQL constitue ainsi un point d'accès unique pour l'analyse, les visualisations et les usages futurs en data science.

2.4.6 Supervision et observabilité : Prometheus et Grafana et Alertmanager

Afin de garantir la fiabilité de la plateforme, un dispositif de supervision basé sur **Prometheus** et **Grafana** et **Alertmanager** a été mis en place.

Prometheus est utilisé pour collecter des métriques techniques telles que :

— l'état des services conteneurisés ;

— la consommation de ressources (CPU, mémoire) ;

— la disponibilité des composants clés (PostgreSQL, Airflow).

Alertmanager assure la centralisation des alertes et l'envoi de notifications par courrier électronique en cas d'incident détecté.

Grafana permet de visualiser ces métriques à travers des tableaux de bord, offrant une vue synthétique de l'état de la plateforme et facilitant l'analyse des anomalies ou des dérives de performance.

Cette approche renforce l'**observabilité** du système.

2.4.7 Python et bibliothèques de collecte

La collecte des données repose sur le langage **Python**, notamment via la bibliothèque `requests` pour l'interrogation des APIs externes. Python s'intègre naturellement avec Spark et Airflow, constituant un langage pivot de la plateforme.

Ce choix est justifié par :

— la richesse de l'écosystème Python orienté données ;

— sa lisibilité et sa maintenabilité ;

— sa forte adoption dans les environnements Big Data.

Chapitre 3

Modélisation et description des tables

La couche **Silver** correspond au niveau de données transformées, structurées et prêtes à être exploitées pour des analyses. Elle repose sur un modèle relationnel normalisé, alimenté de manière incrémentale à partir des données brutes stockées dans la couche Raw.

Quatre tables principales composent cette couche Silver.

3.0.1 Table `dim_city`

La table `dim_city` constitue la dimension de référence des villes. Elle centralise les informations géographiques et d'identification nécessaires aux traitements analytiques.

Chaque ville est identifiée de manière unique par une clé technique `city_id`. Cette table est utilisée comme point d'ancrage pour l'ensemble des données de pollution et de météo.

Principaux attributs :

— identifiant unique de la ville ;
— nom de la ville ;
— pays ;
— latitude et longitude ;
— libellés de ville tels que fournis par les API sources.

3.0.2 Table `air_quality_current`

La table `air_quality_current` contient les mesures de qualité de l'air collectées à un instant donné pour chaque ville.

Chaque enregistrement correspond à un **snapshot horaire** des indicateurs de pollution pour une ville donnée.

Principales informations stockées :

— référence à la ville (`city_id`) ;
— horodatage de la mesure (`measured_at`) ;
— horodatage de collecte (`fetched_at_utc`) ;
— indice global de qualité de l'air (AQI) ;
— polluant dominant ;
— concentrations des principaux polluants (PM10, PM2.5, NO2, O3, SO2, CO) ;
— indicateurs environnementaux associés (température, humidité, pression, vent).

Cette table permet des analyses temporelles fines de l'évolution de la qualité de l'air par ville.

3.0.3 Table `weather_current`

La table `weather_current` stocke les observations météorologiques associées à chaque ville, collectées à la même fréquence horaire.

Chaque ligne correspond à un état météo à un instant donné.

Attributs principaux :
— référence à la ville (`city_id`) ;
— horodatage de la mesure ;
— température, température ressentie, minimum et maximum ;
— pression atmosphérique et humidité ;
— vitesse et direction du vent ;
— couverture nuageuse ;
— description météorologique.

Cette table permet de corréler les données de pollution avec les conditions météorologiques.

3.0.4 Table `air_quality_forecast`

La table `air_quality_forecast` contient les prévisions de qualité de l'air à court terme fournies par l'API WAQI.

Les données sont stockées à une granularité journalière, par type de polluant.

Principaux champs :
— référence à la ville (`city_id`) ;
— date de prévision ;
— type de polluant ;
— valeur minimale, moyenne et maximale prévue ;
— horodatage de collecte.

Cette table permet d'anticiper les tendances de pollution et de comparer prévisions et observations réelles.

3.1 Organisation du modèle analytique

Le modèle de données de la couche Silver s'organise selon une logique proche d'un **schéma en étoile**, adaptée à des besoins analytiques et exploratoires.

3.1.1 Principe du schéma

La table `dim_city` joue le rôle de dimension centrale, commune à l'ensemble des tables de faits.

Les tables de faits sont :
— `air_quality_current` ;
— `weather_current` ;
— `air_quality_forecast`.

Chaque table de faits contient :
— une clé étrangère vers la dimension ville ;
— une dimension temporelle (date ou horodatage) ;
— des indicateurs numériques exploitables.

3.1.2 Granularité des faits

La granularité des tables est clairement définie :
— **air_quality_current** : une ligne par ville et par heure ;
— **weather_current** : une ligne par ville et par heure ;
— **air_quality_forecast** : une ligne par ville, par jour et par polluant.

Cette granularité permet :
— des analyses temporelles fines ;
— des jointures simples entre pollution et météo ;
— une évolution ultérieure vers une couche Gold si nécessaire.

3.1.3 Avantages du modèle

Cette organisation facilite :

— la compréhension du modèle par des profils analystes ;
— l'écriture de requêtes analytiques simples ;
— l'intégration avec des outils de visualisation ;
— la montée en maturité du système décisionnel.

La figure 3.1 illustre l'organisation du modèle analytique retenu pour la couche Silver.

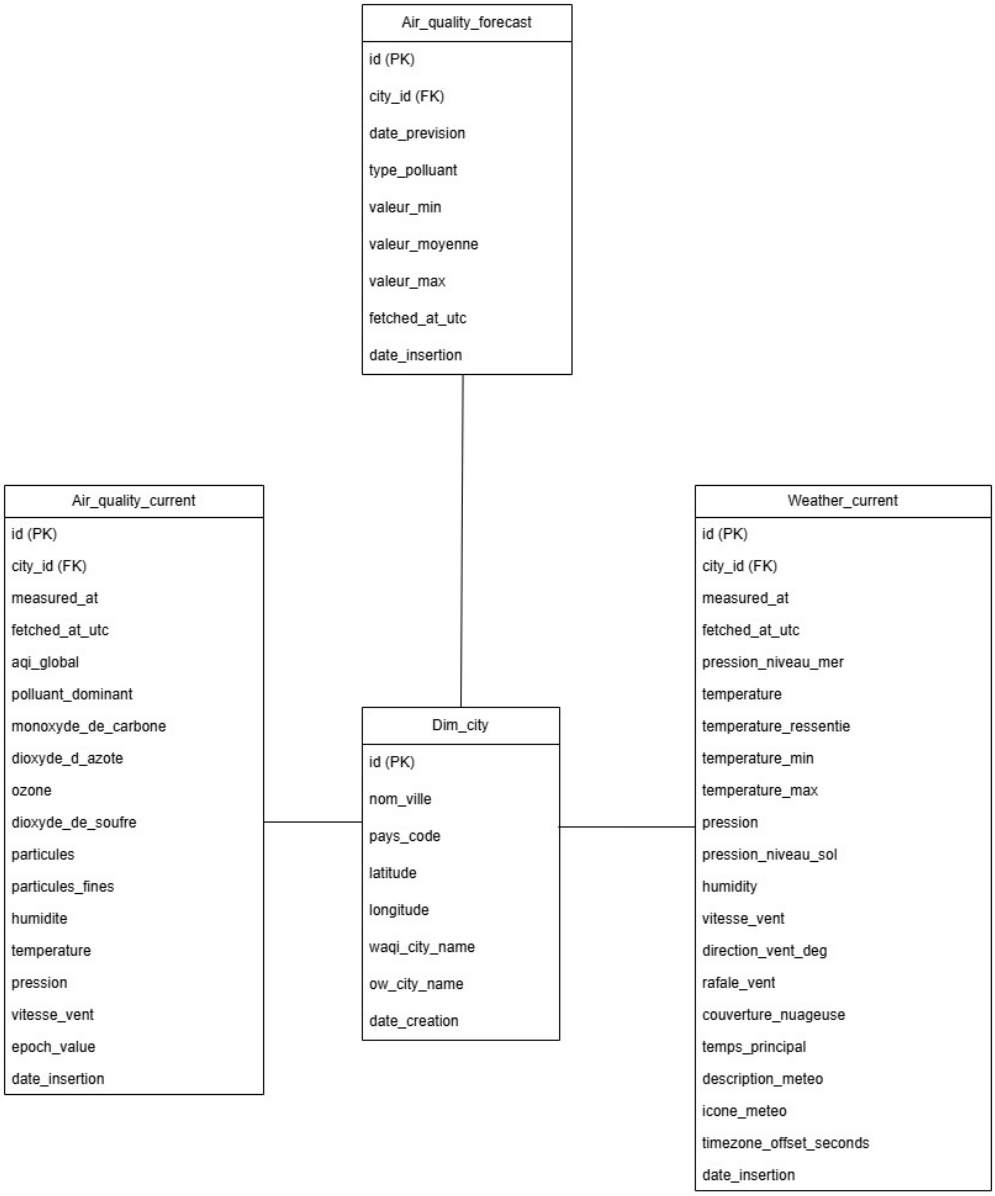

Figure 3.1 – Vue synthétique du modèle analytique de la couche Silver

Chapitre 4

Gestion des logs et supervision du pipeline

La fiabilité d'un pipeline de données repose non seulement sur sa capacité à traiter les données correctement, mais également sur sa capacité à détecter, tracer et gérer les erreurs de manière contrôlée.

Dans le cadre du pipeline GoodAir, une attention particulière a été portée à la gestion des erreurs et à la centralisation des logs afin de garantir :

— la détection rapide des anomalies ;
— la traçabilité complète des traitements ;
— la possibilité de diagnostic et de relance ciblée ;
— la stabilité globale de la plateforme.

4.1 Journalisation des traitements (Logs)

4.1.1 Logs d'orchestration (Airflow)

Apache Airflow constitue le référentiel d'exécution du pipeline `goodair_hourly`. Pour chaque run horaire, Airflow conserve :

— l'horodatage d'exécution et l'identifiant de run ;
— le statut des tâches (*success*, *failed*, *retry*) ;
— la durée d'exécution par tâche et l'ordre des dépendances ;
— les journaux détaillés consultables depuis l'interface web.

Cette journalisation permet de reconstituer l'historique complet des exécutions et d'identifier le point précis de rupture en cas d'échec.

4.1.2 Logs applicatifs : fonctionnels et techniques

Les traitements métiers produisent des logs complémentaires, structurés en deux catégories.

Logs fonctionnels Ils documentent la logique métier et la production de données :

— périmètre traité (source, villes, créneau horaire) ;
— volumes attendus vs obtenus (ex. nombre de villes collectées) ;
— cohérences et contrôles (données manquantes, enregistrements rejetés) ;
— confirmation d'écriture dans la couche Raw et/ou Silver.

Logs techniques Ils facilitent le diagnostic d'infrastructure et d'exécution :

— erreurs réseau / timeout / code HTTP lors des appels API ;

— erreurs de parsing JSON, typage, schéma ;
— erreurs de connectivité (HDFS, JDBC PostgreSQL) ;
— chemins d'entrée/sortie (partitions HDFS, tables PostgreSQL).

4.2 Traçabilité et auditabilité des données

La traçabilité est assurée par :
— le partitionnement temporel strict en couche Raw : `/data/raw/<source>/dt=YYYY-MM-DD/hour=HH/` ;
— l'historique des exécutions Airflow (runs et logs) ;
— des horodatages dans la couche Silver : `measured_at` et `fetched_at_utc`.

Ainsi, chaque donnée Silver peut être rattachée à :
— un créneau horaire de collecte ;
— une partition Raw spécifique ;
— un run Airflow identifiable.

4.3 Métriques et supervision technique (Prometheus / Grafana)

4.3.1 Collecte des métriques : rôle de Prometheus

Prometheus est utilisé comme système de collecte de métriques. Il interroge périodiquement des endpoints (*scrape*) afin d'obtenir des mesures sous forme de séries temporelles.

Dans l'architecture GoodAir, la collecte est configurée notamment sur :
— **Prometheus lui-même** (auto-supervision) ;
— **cAdvisor** pour les métriques des conteneurs Docker (CPU, mémoire, I/O) ;
— **PostgreSQL exporter** pour des métriques liées à la base (connexions, activité) ;
— **StatsD exporter** pour exposer des métriques applicatives compatibles StatsD (ex. agrégation de métriques Airflow si activée).

4.3.2 Exemples de métriques pertinentes

Les métriques suivies répondent à trois objectifs : disponibilité, performance, capacité.

Disponibilité
— état d'accessibilité des services (conteneurs UP/DOWN) ;
— disponibilité de PostgreSQL et des endpoints de métriques.

Performance
— CPU et mémoire consommés durant les traitements ;
— charge et latence sur la base (activité, connexions) ;
— durée et intensité des périodes de calcul (corrélation avec les runs Airflow).

Capacité
— tendances d'utilisation des ressources dans le temps ;
— détection de saturation progressive (mémoire, CPU, disque).

4.3.3 Visualisation : rôle de Grafana

Grafana est utilisé comme couche de visualisation. Il se connecte à Prometheus comme source de données et permet :
— la construction de tableaux de bord (vue globale, vue base, vue conteneurs) ;
— l'analyse temporelle des charges et la mise en évidence de dérives ;
— la corrélation entre charges système et traitements planifiés.

La figure 4.1 présente un exemple de tableau de bord Grafana utilisé. Ce tableau de bord permet de visualiser en temps réel l'état des services supervisés et d'identifier rapidement toute anomalie ou dérive de performance.

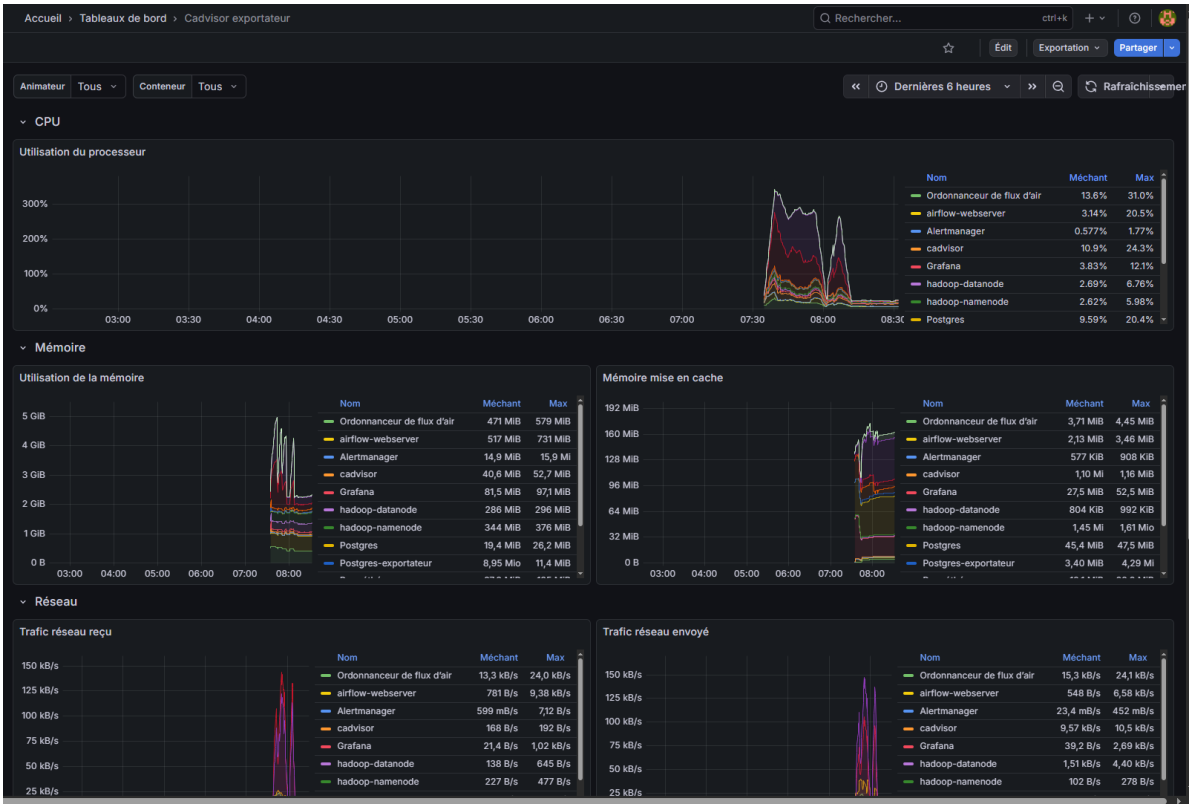

Figure 4.1 – Exemple de tableau de bord Grafana pour la supervision du pipeline GoodAir

4.4 Alerting et gestion proactive des incidents

L'alerting repose sur l'exploitation des métriques techniques exposées par Airflow via le protocole StatsD, collectées par Prometheus puis analysées à l'aide de règles d'alerte dédiées.

4.4.1 Règle d'alerte sur les échecs de tâches Airflow

Une règle d'alerte spécifique a été définie afin de détecter l'échec d'au moins une tâche appartenant au DAG `goodair_hourly` sur une fenêtre temporelle récente.

Cette règle s'appuie sur les métriques de fin d'exécution des tâches Airflow, en particulier les métriques indiquant un statut *failed*. Lorsqu'au moins une tâche échoue sur une période de cinq minutes, une alerte est déclenchée après une temporisation d'une minute afin d'éviter les faux positifs liés à des erreurs transitoires.

4.4.2 Rôle d'Alertmanager

Les alertes générées par Prometheus sont transmises à Alertmanager, qui assure la gestion et la diffusion des notifications. Dans le cadre du projet GoodAir, Alertmanager est configuré pour envoyer une notification par courrier électronique lorsqu'une alerte critique est déclenchée.

Cette notification contient notamment :

— le niveau de sévérité de l'alerte ;
— le nom du pipeline concerné ;
— l'identifiant du DAG Airflow impacté ;
— une description synthétique de l'incident détecté.

Ce mécanisme permet une détection rapide des échecs de traitements et facilite l'intervention manuelle pour l'analyse des causes et la relance ciblée du pipeline.

Conclusion

Synthèse des réalisations et conformité au Bloc 3 (TPRE843)

Ce rapport a formalisé la conception et la structuration de la plateforme Big Data **GoodAir**, en répondant aux exigences de la MSPR **TPRE843** (Bloc 3), centrées sur la mise en place d'une plateforme de données **robuste**, **traçable** et **sécurisée**.

La solution retenue repose sur une architecture **ELT** conteneurisée (**Docker**), orchestrée par **Apache Airflow** et s'appuyant sur **Apache Spark** pour le traitement distribué. Elle permet de valider les compétences clés attendues :

1. **Architecture de stockage et performance** : une architecture de stockage duale a été mise en place avec un **Data Lake HDFS** pour l'archivage des données brutes (traçabilité et rejouabilité) et un **entrepôt unique PostgreSQL** pour centraliser les données structurées. La modélisation normalisée de la couche Silver, proche d'un schéma en étoile, garantit des requêtes analytiques efficaces sur des volumes de données croissants.
2. **Pipeline et fiabilité des données** : le pipeline **horaire** piloté par Airflow assure l'historisation incrémentale des mesures à **H :00**. Cette organisation permet de définir et déployer des flux d'intégration par lot (*micro-batch*) en prenant en compte les contraintes externes, notamment le respect des quotas imposés par les APIs publiques.
3. **Sécurité et qualité** : l'isolation des composants via Docker, l'authentification sur les points d'accès (PostgreSQL, Airflow) et la prise en compte d'un hébergement compatible UE constituent les fondations de sécurité de la plateforme. Par ailleurs, un dispositif de supervision par **métriques** (**Prometheus**/**Grafana**) peut être mobilisé pour observer l'état des services et les performances du pipeline. *L'alerting automatisé par mail en cas d'échec.*

Ouverture et évolutivité du système

L'architecture GoodAir a été conçue pour être **évolutive** et alignée avec une démarche d'industrialisation progressive. Conformément à une approche itérative, une première itération (**MVP**) a permis de déployer le socle technique, de valider l'ingestion en couche Raw et de poser les bases de l'observabilité (logs et métriques).

Les évolutions futures s'inscrivent naturellement dans la continuité des objectifs du **Bloc 5** :

— les données propres et historisées centralisées dans PostgreSQL sont disponibles pour la production de rapports, d'indicateurs et de visualisations via un outil BI ;

— la présence d'un Data Lake HDFS et la capacité de traitement distribué via Spark permettent d'envisager des analyses avancées et l'intégration de modèles statistiques ou de *machine learning*, notamment pour des usages de prédiction (par exemple la détection d'épisodes de canicule ou l'anticipation de tendances de pollution).

Ainsi, la plateforme **GoodAir Data Platform** constitue une fondation technique cohérente et pérenne, destinée à accompagner le laboratoire TotalGreen dans ses ambitions scientifiques et stratégiques.

Annexe – Liens vers les ressources du projet

Cette annexe regroupe les liens vers les principales ressources externes utilisées dans le cadre du projet *GoodAir*, incluant les sources de données, le dépôt de code et l'outil de gestion de projet.

Sources de données

- **World Air Quality Index (WAQI)** :
  https://waqi.info
- **OpenWeather API** :
  https://openweathermap.org/api

Gestion de version – GitLab

- Dépôt GitLab du projet GoodAir :
  https://gitlab.com/Karlito2.0/mspr

Gestion de projet – Trello

- Tableau Trello de suivi du projet GoodAir :
  https://trello.com/b/7QpkMPTy/mspr